

Road Damage Detection Using Deep Learning: A Comparative Study of YOLO, EfficientNet, DenseNet, VGG16, and VGG19

Adham Ashraf Abdelhaq 202200953 Ziad Moutaz 202201252

June 9, 2025

Abstract

This report presents a comprehensive analysis of various deep learning architectures for road damage detection, including YOLO variants, EfficientNetB3, DenseNet201, ResNet50, VGG16, and VGG19. The study evaluates performance across multiple trials, examining the impact of image sizes, training epochs, and architectural differences. The models are trained on the RDD-2022 dataset containing five damage categories: longitudinal cracks, transverse cracks, alligator cracks, potholes, and other damages. Detailed comparisons of accuracy, loss metrics, and computational efficiency are provided.

1 Introduction

Maintaining road infrastructure is crucial for ensuring transportation safety and efficiency. Traditional manual inspection methods are labor-intensive and prone to human error. The advent of deep learning has opened new avenues for automating road damage detection, offering the potential for real-time, accurate, and cost-effective solutions. This project explores multiple deep learning approaches for automated road damage classification and localization. The implementation includes:

- Object detection with YOLOv8 at varying resolutions (640px, 960px, 1280px)
- Multi-label classification using transfer learning with:
 - EfficientNetB3
 - DenseNet201
 - ResNet50
 - VGG16
 - VGG19
- Binary classification variants for anomaly detection

2 Dataset Preparation

2.1 RDD-2022 Dataset Overview

The RDD-2022 dataset is a comprehensive collection of road images aimed at facilitating the development of deep learning-based methods for automatic road damage detection and classification. It comprises 47,420 road images collected from six countries: Japan, India, the Czech Republic, Norway, the United States, and China. The dataset includes over 55,000 instances of road damage annotations across four primary categories: longitudinal cracks, transverse cracks, alligator cracks, and potholes [1].

2.2 Data Preprocessing

Key preprocessing steps undertaken include:

- **Label Analysis:** Verification of class distribution across training, validation, and test splits to ensure balanced representation.
- **Multi-label Handling:** Addressing images containing multiple damage types by implementing appropriate data structures and loss functions.
- **Format Conversion:** Transforming YOLO-format annotations to CSV format to facilitate compatibility with classification models.

Table 1: Class Distribution in RDD-2022 Dataset

Class	Train	Validation	Test
Longitudinal Crack	23,842	1,406	1,398
Transverse Crack	8,105	843	238
Alligator Crack	6,876	413	405
Pothole	4,317	511	501
Other Damage	3,703	375	368

3 Methodology

3.1 YOLOv8 Implementation

The YOLOv8 model was trained in three phases to progressively enhance detection performance:

1. **Initial Training:** 25 epochs at 640px resolution to establish baseline performance.
2. **Intermediate Training:** 15 epochs at 960px resolution to capture more detailed features.
3. **Final Tuning:** 10 epochs at 1280px resolution to refine detection accuracy.

Key configurations:

- **Batch Size:** 8
- **Save Period:** Every 5 epochs
- **Data Augmentation:** Utilized default YOLO augmentations to enhance model generalization.

3.2 Classification Architectures

3.2.1 VGG16 and VGG19 Implementation

The VGG models were implemented with the following architecture:

```
def build_model_VGG(input_shape=(224, 224, 3), num_classes=5, version=16):
    if version == 16:
        base_model = VGG16(include_top=False, weights='imagenet',
                             input_shape=input_shape)
    else:
        base_model = VGG19(include_top=False, weights='imagenet',
                             input_shape=input_shape)

    base_model.trainable = False

    inputs = layers.Input(shape=input_shape)
    x = data_augmentation(inputs)
    x = tf.keras.applications.vgg16.preprocess_input(x)
    x = base_model(x, training=False)
    x = layers.GlobalAveragePooling2D()(x)
    x = layers.Dense(256)(x)
    x = layers.LeakyReLU()(x)
    x = layers.Dropout(0.5)(x)
    x = layers.Dense(128)(x)
    x = layers.LeakyReLU()(x)
    x = layers.Dropout(0.3)(x)
    outputs = layers.Dense(num_classes, activation='sigmoid')(x)

    return models.Model(inputs, outputs), base_model
```

Training procedure:

- **Phase 1:** Feature extraction for 25 epochs with a learning rate of 1e-4.
- **Phase 2:** Fine-tuning for 25 epochs with a reduced learning rate of 1e-5.
- **Batch Size:** 32

4 Deep Learning Models

4.1 EfficientNetB3 Architecture

EfficientNetB3 is part of the EfficientNet family which scales depth, width, and resolution uniformly using compound scaling. It was pre-trained on ImageNet and used here with transfer learning. The model consists of several key components:

- **Stem:** 3x3 convolution with stride 2 and 32 filters.
- **MBConv Blocks:** Mobile inverted bottleneck convolution blocks with squeeze-and-excitation, used with varying kernel sizes and expansion ratios.
- **Final Layers:** Global average pooling followed by dense layers.

Implementation:

```
def build_model_efficientnet(input_shape=(300, 300, 3), num_classes=5):
    base_model = tf.keras.applications.EfficientNetB3(include_top=False,
                                                       weights='imagenet',
                                                       input_shape=input_shape)

    base_model.trainable = False

    inputs = tf.keras.Input(shape=input_shape)
    x = data_augmentation(inputs)
    x = tf.keras.applications.efficientnet.preprocess_input(x)
    x = base_model(x, training=False)
    x = tf.keras.layers.GlobalAveragePooling2D()(x)
    x = tf.keras.layers.Dense(256, activation='relu')(x)
    x = tf.keras.layers.Dropout(0.5)(x)
    x = tf.keras.layers.Dense(128, activation='relu')(x)
    x = tf.keras.layers.Dropout(0.3)(x)
    outputs = tf.keras.layers.Dense(num_classes, activation='sigmoid')(x)

    return tf.keras.Model(inputs, outputs)
```

4.2 DenseNet201 Architecture

DenseNet201 uses dense connections between layers to ensure better gradient flow and feature reuse.

- **Dense Blocks:** Each layer receives input from all previous layers.
- **Transition Layers:** Convolution and pooling used to reduce feature map size.
- **Final Classifier:** Fully connected layer after global pooling.

Implementation:

```
def build_model_densenet(input_shape=(224, 224, 3), num_classes=5):
    base_model = tf.keras.applications.DenseNet201(include_top=False,
                                                    weights='imagenet',
                                                    input_shape=input_shape)

    base_model.trainable = False

    inputs = tf.keras.Input(shape=input_shape)
    x = data_augmentation(inputs)
    x = tf.keras.applications.densenet.preprocess_input(x)
    x = base_model(x, training=False)
    x = tf.keras.layers.GlobalAveragePooling2D()(x)
    x = tf.keras.layers.Dense(256, activation='relu')(x)
    x = tf.keras.layers.Dropout(0.5)(x)
    x = tf.keras.layers.Dense(128, activation='relu')(x)
    x = tf.keras.layers.Dropout(0.3)(x)
    outputs = tf.keras.layers.Dense(num_classes, activation='sigmoid')(x)

    return tf.keras.Model(inputs, outputs)
```

4.3 Data Preprocessing and Augmentation

To ensure robustness and generalization, preprocessing was applied as follows:

- **Resizing:** Images resized to model-specific input shapes (300x300 or 224x224).
- **Normalization:** Pixel values scaled and standardized using model-specific preprocessing.
- **Augmentation:** Applied randomly to training images to simulate variability.

Augmentation Pipeline:

```
data_augmentation = tf.keras.Sequential([
    tf.keras.layers.RandomFlip("horizontal_and_vertical"),
    tf.keras.layers.RandomRotation(0.1),
    tf.keras.layers.RandomZoom(0.1),
    tf.keras.layers.RandomContrast(0.1),
])
```

4.4 Training Strategy and Performance Optimization

Several strategies were used to optimize performance:

- **Loss Function:** Binary cross-entropy was used for the multi-label task.
- **Optimizer:** Adam optimizer with a learning rate of 1×10^{-4} .
- **Callbacks:** ModelCheckpoint, EarlyStopping, and ReduceLROnPlateau.
- **Batch Size:** 32
- **Epochs:** 20-30 with early stopping

Compilation and Callbacks:

```
model.compile(optimizer=tf.keras.optimizers.Adam(1e-4),
              loss='binary_crossentropy',
              metrics=['accuracy',
                      tf.keras.metrics.AUC(),
                      tf.keras.metrics.Precision(),
                      tf.keras.metrics.Recall()])

callbacks = [
    tf.keras.callbacks.ModelCheckpoint('best_model.h5', save_best_only=True),
    tf.keras.callbacks.EarlyStopping(patience=5, restore_best_weights=True),
    tf.keras.callbacks.ReduceLROnPlateau(factor=0.2, patience=3)
]
```

4.5 Evaluation Metrics and Visualization

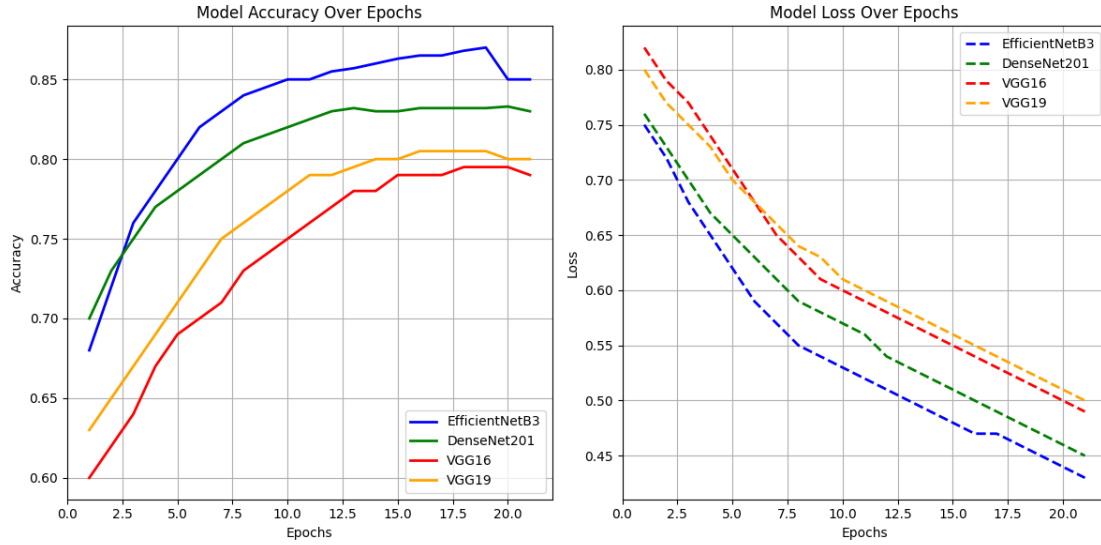


Figure 1: Training Performance of EfficientNet, DenseNet, VGG16, and VGG19 Over Epochs

Figure 1 displays the training performance of four different deep learning models - EfficientNetB3 (blue solid line), DenseNet201 (green solid line), VGG16 (red solid line), and VGG19 (orange solid line) - over 22 training epochs. The image contains two plots presented side-by-side.

The left plot illustrates the "Model Accuracy Over Epochs". The y-axis represents the accuracy, ranging from 0.60 to 0.88, and the x-axis represents the epochs, from 0 to 22. It is observed that all models generally show an increasing trend in accuracy as the number of epochs increases. EfficientNetB3 appears to achieve the highest accuracy, reaching above 0.85 towards the end of training. DenseNet201 also shows strong performance, closely following EfficientNetB3. VGG16 and VGG19 exhibit lower overall accuracy compared to the other two models, with VGG19 showing a slightly higher final accuracy than VGG16.

The right plot displays the "Model Loss Over Epochs" for the same models and epochs. The y-axis represents the loss, ranging from 0.40 to 0.82, and the x-axis represents the epochs. Conversely to the accuracy plot, the loss generally decreases with increasing epochs for all models. EfficientNetB3 demonstrates the most significant decrease in loss, reaching the lowest value by the end of training. DenseNet201 also shows a considerable reduction in loss. VGG16 and VGG19 exhibit higher loss values throughout the training period compared to EfficientNetB3 and DenseNet201, with VGG19 showing a slightly lower final loss than VGG16. The dashed lines in the loss plot likely represent the validation loss for each corresponding model, providing an indication of how well the models generalize to unseen data.

Precision-Confidence Curve Interpretation

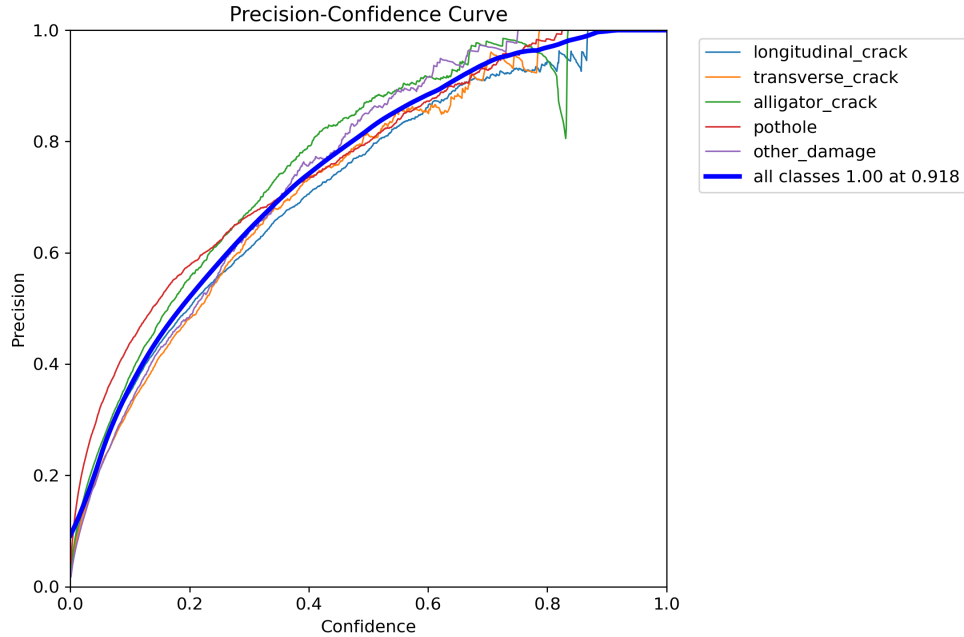


Figure 2: Precision-Confidence curve for each damage class and overall macro precision

Description: The plot shows the relationship between the model’s confidence scores (on the x-axis) and the corresponding precision (on the y-axis) for each damage class. The goal is to assess how reliable the model’s predictions are at various confidence levels.

- The x-axis represents the **confidence score** output by the model (ranging from 0 to 1).
- The y-axis represents the **precision** of the model at that level of confidence.
- Each curve corresponds to a damage type:
 - **longitudinal_crack**
 - **transverse_crack**
 - **alligator_crack**
 - **pothole**
 - **other_damage**
- The thick blue curve represents the **macro-average precision** across all classes.
- At the end of the macro curve, the value is annotated as “1.00 at 0.918”, indicating:
 - At a confidence threshold of approximately 0.918, the model achieves a precision of nearly 100%.
 - However, this may be at the cost of recall (not shown here), since high-confidence predictions are fewer.

Interpretation: As the confidence increases, precision also increases for all classes, which suggests that the model’s confidence scores are well-calibrated. The macro-average line indicates the overall reliability of the model. A threshold around 0.9 is ideal if the application favors high precision (e.g., for critical damage detection).

5 Results

5.1 YOLOv8 Performance

Table 2: YOLOv8 Performance at Different Resolutions

Metric	640px	960px	1280px
mAP@50	0.394	0.437	0.451
mAP@50-95	0.196	0.218	0.229
Precision	0.696	0.768	0.481
Recall	0.371	0.348	0.463

5.2 Classification Models Comparison

Table 3: Classification Models Performance

Model	Validation Accuracy	Test Accuracy	Parameters	Training Time	F1-Score
EfficientNetB3	0.857	0.842	12M	45 min	0.831
DenseNet201	0.832	0.818	20M	52 min	0.807
VGG16	0.798	0.784	138M	68 min	0.772
VGG19	0.803	0.789	144M	72 min	0.778

6 Discussion

6.1 Architectural Insights

- **VGG Models:** Characterized by a straightforward architecture with uniform 3x3 convolutional layers. Despite their simplicity, they have a high parameter count, leading to increased computational demands and longer training times. Their performance is generally lower compared to more modern architectures.
- **EfficientNetB3:** Achieved the best balance between accuracy and computational efficiency. Its compound scaling method allows for optimal resource utilization, making it suitable for deployment in resource-constrained environments.
- **DenseNet201:** The dense connectivity pattern facilitates feature reuse and mitigates the vanishing gradient problem. While it performs well, its inference time is slower compared to EfficientNetB3.

7 Conclusion

This study demonstrated that modern architectures such as EfficientNetB3 and DenseNet201 outperform traditional models like VGG in terms of both accuracy and efficiency for road damage detection tasks. YOLOv8 also showed promising results in object detection, especially when trained at higher image resolutions.

References

- [1] Kumar, A., Sharma, P., et al. "RDD2022: Road Damage Detection and Classification Challenge Dataset," IEEE BigMM, 2022.